Authentication & Authorization



Authentication vs Authorization

Authentication is the process of verifying the *identity* of an individual, device, or entity attempting to access a system, network, or resource.

Authorization is the process of granting or denying *access* to specific resources or functionalities based on the identity and privileges of an individual or entity.

DRIM

Authentication

Stateful Authentication

Stateful authentication involves maintaining a *session state* on the server side to identify and track authenticated users. The session state typically includes data such as the user's identity, permissions, and other relevant details. When a user logs in successfully, the server creates a session and assigns a session identifier (e.g., a session token or session ID) to the client. This session identifier is typically stored in a cookie or included in the request headers. On subsequent requests, the client sends the session identifier to the server, allowing the server to look up the session and verify the user's identity.

Stateful Authentication

Pros:

- The server has fine-grained control over user sessions
- Session data can be used to store additional user-related information
- Session expiration can be managed for improved security
- Session invalidation is straightforward

Stateful Authentication

Cons:

- Server resources are consumed to store and manage session data
- Extra state access for each request
- Scaling can be challenging as the session state must be synchronized across multiple servers
- High availability and failover mechanisms become more complex
- Stateless server architecture is not feasible

Stateless Authentication

Stateless authentication, as the name suggests, does not rely on a server-side session state. Instead, each request from the client must contain all the necessary authentication information for the server to verify the user's identity. The server *independently* validates each request without referencing previous interactions or maintaining any session-related information.

Stateless Authentication

Pros:

- Scalability is improved as servers do not need to synchronize session state
- Server resources are not consumed for storing session data
- Stateless server architecture is possible, enabling easy scaling and load balancing
- Simplified server logic as each request is self-contained

Stateless Authentication

Cons:

- No inherent session management or revocation mechanism
- The client must include authentication information in each request, which may increase request size
- Additional security measures might be needed to handle token-based authentication securely
- Difficulty in handling situations where session invalidation or revocation is required

DRIM

Authorization

Authorization Types

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)
- Discretionary Access Control (DAC)
- Mandatory Access Control (MAC)

Role-Based Access Control

RBAC is a widely used authorization model that grants access based on predefined roles. Users are assigned specific *roles*, and access rights are associated with these roles. RBAC simplifies access management by grouping permissions and roles, making it easier to assign and revoke access based on job responsibilities.

Attribute–Based Access Control

ABAC is a flexible authorization model that grants access based on the evaluation of *attributes (claims)* associated with users, resources, and the environment. These attributes can include user characteristics, resource properties, and contextual information. Policies define the access rules based on the attribute values.

Discretionary Access Control

In a **DAC** model, each resource has an owner who has the authority to set access permissions for that resource. The owner can determine which users or groups are allowed to access the resource and specify the type of access they have, such as read, write, or execute permissions. The access control decisions are discretionary because they are at the discretion of the owner. These decisions are often in the form of *access control lists (ACLs)*.

A prominent example of DAC is Unix file system access control.

Mandatory Access Control

MAC is a strict authorization model used in high-security environments. Access decisions are based on labels assigned to subjects (users) and objects (resources). The labels define the sensitivity and integrity levels, and access is granted or denied based on predefined rules.

DRIM

Access Token & Refresh Token

Access Token

Access token is a credential that is issued by an authentication server after a user successfully authenticates an application. It is used to access protected resources (API endpoints, services, etc.) on behalf of the user or application. The access token is typically short-lived and has an expiration time associated with it. Once the access token expires, it can no longer be used to access resources.

Bearer Token

Bearer tokens are called "bearer" because they are presented by the client application to the server to gain access to resources, without the need for additional credentials or proof of identity. The bearer token itself is typically a string of characters, often encoded in JSON Web Token (JWT) format and is included in the request headers or as a parameter in API calls.

Bearer access tokens are used for *stateless authentication*.

Refresh Token

Refresh token is a long-lived credential that is also issued by the authentication server alongside the access token. The purpose of the refresh token is to obtain a new access token once the previous one expires, without requiring the user to re-authenticate. Refresh tokens have a longer expiration time compared to access tokens. When an access token expires, the application can use the refresh token to request a new access token from the authentication server.

Refresh tokens are used for *stateful authentication*.

Access Token & Refresh Token

The combined usage of bearer access tokens and refresh tokens present a compromise between *stateless* and *stateful* authentication:

- Most requests do not need to perform state access
- In case of an access token leak, the time window for unauthorized access is short
- Access can be revoked by revoking a refresh token
- Users do not need to re-authenticate using their credentials

DRIM

JWT Access Tokeb

JWT stands for JSON Web Token. It is an open standard (RFC 7519) that defines a compact and self-contained way of transmitting information between parties as a JSON object. JWTs are commonly used for authentication and authorization purposes in web applications and APIs.

JWT Structure

A JWT token consists of three parts separated by dots:

- **Header**. It contains information about the type of token and the cryptographic algorithm used to sign it
- **Payload**. It contains the claims or statements about the authenticated entity (user) and additional metadata. Claims can be either registered, public, or private
- **Signature**. The signature is created by combining the encoded header, encoded payload, and a secret key known only to the server. It is used to verify that the token has not been tampered with during transmission and ensures its authenticity. The signature is calculated by applying the specified algorithm (mentioned in the header) to the encoded header and payload along with the secret key

JWT Sample

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93c
y8yMDA1LzA1L2lkZW50aXR5L2NsYW1tcy9uYW1l
Ijoic2VsbGVyNSIsImh0dHA6Ly9zY2hlbWFzLm1
pY3Jvc29mdC5jb20vd3MvMjAwOC8wNi9pZGVudG
l0eS9jbGFpbXMvcm9sZSI6WyJTZWxsZXIiLCJCd
XllciJdLCJyYW5rIjoiNSIsImV4cCI6MTY4NTAw
NTE4NiwiaXNzIjoiY3J5cHRvLWJhbmsiLCJhdWQ
iOiJjcnlwdG8tYmFuayJ9.3DfuO-
MpkVtymhCvtQTHim9NhETl-llLNAdbHgWlCXY
```

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{

"http://schemas.xmlsoap.org/ws/2005/05/identity/claims/
name": "seller5",

"http://schemas.microsoft.com/ws/2008/06/identity/claim
s/role": [
    "Seller",
    "Buyer"
  ],
  "rank": "5",
  "exp": 1685005186,
  "iss": "crypto-bank",
  "aud": "crypto-bank"
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) secret base64 encoded
```

DRIM

Access & Refresh Tokens Flow

Operations

- Authenticate
- RefreshToken
- RevokeRefreshToken

Authenticate

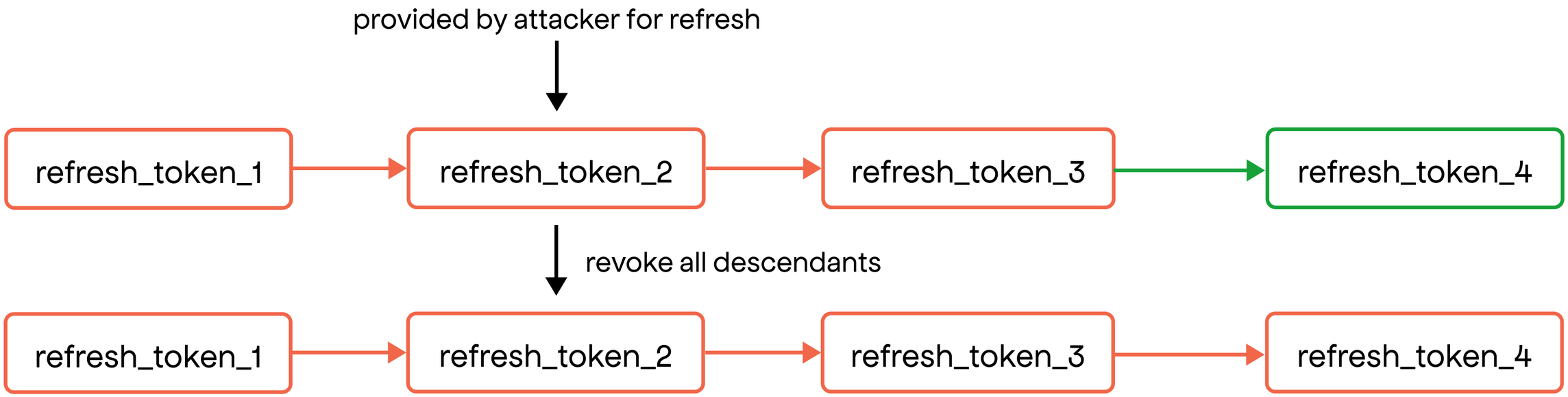
- User provides credentials (username and password), application returns pair <access_token, refresh_token>
- access_token is JWT and is not stored in the DB
- refresh_token is an opaque string and is stored in the DB for the user
- User's old refresh tokens are deleted from the DB

RefreshToken

- User provides refresh_token to get a new pair <access_token, refresh_token>
- If provided refresh_token not found in the DB, fail
- If provided refresh_token is revoked, revoke all descendant refresh tokens from the chain, alert support, and fail
- If refresh_token expired, fail
- Generate a new pair <refresh_token, access_token>
- Store new refresh_token in ReplacedBy property of old refresh_token to form a chain
- Return the new pair <access_token, refresh_token>

The described process is called *Refresh Token Rotation (RTR)*

RefreshToken



RevokeRefreshToken

- Admin provides refresh_token to be revoked
- If provided refresh_token not found in the DB, fail
- If provided refresh_token is already revoked, fail
- If provided refresh_token expired, fail
- Mark refresh_token as revoked and update in the DB

DRIM

Password Storage

Plain Text

Store password as is

Problem:

Storing passwords in plain text is a major security risk and is strongly discouraged. If passwords are stored in plain text and a data breach occurs, attackers can easily access and misuse those passwords, potentially compromising user accounts and their associated information. Additionally, users often reuse their passwords, so other services can be compromised too.

Password Hash

Hash password with something like SHA-256 and store the hash

Problem:

After a data breach, attackers can use *rainbow tables* to disclose passwords. A rainbow table is a precomputed table used in password cracking to reverse hash functions and quickly find the original plaintext password from its hash value.

Password + Salt Hash

Add random salt to the password and hash the resulting string with SHA–256. Store the salt alongside the hash

Problem:

After a data breach, attackers can use a dictionary and hash all its entries with the stored salt to find the password. Since SHA–256 hash can be computed very quickly this process will not take a lot of time.

We need a stronger and slower hash function.

Password + Salt Hash with Argon2

Add random salt to the password and hash them with Argon2. Store the salt and Argon2 params alongside the hash

Input parameters: password, salt, memory cost (the memory usage of the algorithm), time cost (the execution time of the algorithm and the number of iterations), parallelism factor (the number of parallel threads), hash length.

MyPassword

↓

$argon2id$v=19$m=32768,t=4,p=1$AVAUSRalUFlTWhdQsRStqg$/xzN4dPelc3uFMiTX7ksXtB3Jc6+gQxLZiNPyPwfyvU

Alternatives: Bcrypt, Scrypt. PBKDF2 is considered obsolete.

Thank you! Questions?

